

UNIVERSIDAD TÉCNICA ESTATAL DE
QUEVEDO

Facultad de Ciencias de la Ingeniería
Carrera de Ingeniería de Software

ASIGNATURA: Aplicaciones Web
Quinto Nivel — Período Académico 2026-2027

Especificación de Requisitos de
Software (SRS)

Estándar ISO/IEC/IEEE 29148:2018

Sistema de Simulación de Comportamiento
Vial con Inteligencia Artificial (SBVIA)

Versión 1.0.0

Repositorio	https://github.com/keithdrox/SBVIA-AppWeb
Punto del historial	etiqueta v1.0.0
DOI del software (Zenodo)	10.5281/zenodo.22740480
Fecha de emisión	17 de septiembre de 2026

AUTORES

Cruz Pérez, Justyn K.
ORCID: 0009-0009-0847-0780
Correo: jcruzp@uteq.edu.ec

Umaginga Arévalo, Jefferson M.
ORCID: 0009-0005-9831-0801
Correo: jumagingaa@uteq.edu.ec

DOCENTE-DIRECTOR

Ing. Gleiston Cíceron Guerrero Ulloa, Ph.D.

Índice

Control del documento	2
1. Introducción	3
1.1. Propósito	3
1.2. Alcance	3
1.3. Audiencia	3
1.4. Definiciones	3
2. Descripción general	4
2.1. Perspectiva del producto	4
2.2. Clases de usuario	4
2.3. Restricciones	4
2.4. Suposiciones y dependencias	4
3. Requisitos funcionales	5
4. Requisitos no funcionales	8
5. Historias de usuario (Casos de Uso)	11
6. Criterios de aceptación de la versión	13

Control del documento

Versión	Fecha	Descripción	Estado
1.0.0	17-09-2026	Línea base de la entrega final	Pendiente de aprobación

1. Introducción

1.1. Propósito

Este documento establece los requisitos verificables de SBVIA v1.0.0 y constituye la línea base contractual para desarrollo, prueba, despliegue y evaluación académica.

1.2. Alcance

SBVIA es una aplicación web de formación vial que administra usuarios y escenarios, conserva prácticas de simulación, calcula resultados y presenta retroalimentación. Incluye un backend Java 21 con Spring Boot, un frontend Angular, PostgreSQL 16, Redis 7 y orquestación Docker Compose.

1.3. Audiencia

Equipo de desarrollo, docente-director, evaluadores, administradores, instructores y conductores en formación.

1.4. Definiciones

Término	Definición
CRUD-ORM	Operaciones elementales implementadas mediante Spring Data JPA.
SP	Procedimiento almacenado o función SQL parametrizada.
JWT	Token firmado para autenticación stateless.
SUS	System Usability Scale.
p95	Percentil 95 de una distribución de latencias.
VERIFIED	El requisito ha sido implementado y comprobado mediante pruebas automatizadas exhaustivas o auditorías.
TESTED	El requisito ha sido verificado únicamente a nivel de componente o prueba superficial, o implementado exitosamente.
IMPLEMENTED	El requisito está desarrollado en el código fuente, pero su cobertura de prueba no valida completamente el criterio.
Inactividad	Ausencia de autenticación y uso del sistema por un período superior a 6 meses.
Práctica	Registro individual del desempeño de un conductor en un escenario específico.
Certificado	Documento inmutable emitido al aprobar un escenario.
Puntaje	Valor numérico de 0 a 100 asignado por la simulación.
Escenario	Configuración específica del entorno vial (clima, tráfico, dificultad) a ser simulado.
Infracción	Violación a una regla de tránsito registrada durante una práctica.
Comportamiento Vial	Decisión tomada por el usuario evaluada por el motor de IA.

2. Descripción general

2.1. Perspectiva del producto

La solución adopta arquitectura cliente-servidor. Angular consume una API REST protegida por JWT; Spring Boot implementa lógica, autorización por roles y persistencia; PostgreSQL conserva los datos y Redis gestiona caché y revocación de tokens.

2.2. Clases de usuario

Rol	Capacidades principales
Conductor	Registro, autenticación, consulta de escenarios, prácticas, métricas y perfil.
Administrador	Gestión de escenarios, usuarios, roles y consulta global de prácticas.
Instructor	Consulta de resultados agregados de estudiantes a su cargo.

2.3. Restricciones

La versión estable utiliza Java 21, Spring Boot, Angular 17 o superior, PostgreSQL 16, Flyway, Redis 7 y Docker Compose. No se admiten secretos productivos versionados. El despliegue público debe usar HTTPS.

2.4. Suposiciones y dependencias

El navegador admite cookies SameSite y JavaScript moderno. El entorno dispone de Docker y conectividad entre servicios. Los servicios externos de publicación, DOI y dominio son responsabilidad organizativa y no se simulan dentro del software.

3. Requisitos funcionales

RF-01 - Autenticación stateless con JWT
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: CRUD-ORM
Descripción: El sistema deberá autenticar a los usuarios mediante credenciales válidas y emitir un JWT (JSON Web Token) stateless.
Criterio de aceptación: Dado un usuario registrado, cuando presenta credenciales válidas, entonces recibe un JWT vigente y cookies seguras; credenciales inválidas producen 401.
Verificación: Prueba de Integración (AuthServiceTest.java) SQL: No aplica

RF-02 - Gestión de Escenarios viales (CRUD)
Tipo: FUNC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: CRUD-ORM
Descripción: El sistema deberá permitir a los administradores crear, leer, actualizar y eliminar (desactivar) escenarios viales, restringiendo la escritura a los conductores.
Criterio de aceptación: Un administrador puede crear, consultar, actualizar y desactivar escenarios; un conductor solo puede consultarlos y recibe 403 al intentar modificarlos.
Verificación: Prueba de Integración (EscenarioServiceTest.java) SQL: No aplica

RF-03 - Consulta multi-tabla de resultados de simulación
Tipo: FUNC Prioridad: Must Estado: IMPLEMENTED
Módulo: Backend Acceso a datos: SP
Descripción: El sistema deberá recuperar los resultados detallados de una simulación consultando múltiples tablas mediante un procedimiento almacenado.
Criterio de aceptación: Al consultar una simulación existente, el sistema compone sus datos relacionados mediante el procedimiento <code>sp_reporte_simulacion</code> y retorna un resultado consistente.
Verificación: Prueba Unitaria (SimulacionServiceTest.java) SQL: db/procs/sp_reporte_simulacion.sql

RF-04 - Cálculo agregado de promedio de desempeño de conductor
Tipo: FUNC Prioridad: Must Estado: IMPLEMENTED
Módulo: Backend Acceso a datos: SP
Descripción: El sistema deberá calcular el promedio global de desempeño de un conductor evaluando todas sus simulaciones completadas.
Criterio de aceptación: Para un conductor con simulaciones, el promedio calculado por el procedimiento <code>sp_calcular_promedio_usuario</code> coincide con los datos persistidos.
Verificación: Prueba Unitaria (SimulacionServiceTest.java) SQL: db/procs/sp_calcular_promedio_usuario.sql

RF-05 - Generación de reporte consolidado diario de actividad			
Tipo: FUNC	Prioridad: Should	Estado: IMPLEMENTED	
Módulo: Backend Acceso a datos: SP			
Descripción: El sistema deberá generar un reporte diario que consolide toda la actividad del sistema para la fecha especificada.			
Criterio de aceptación: El reporte diario incluye la actividad correspondiente a la fecha solicitada.			
Verificación:	Prueba Unitaria	(SimulacionServiceTest.java)	SQL:
db/procs/sp_reporte_actividad_diaria.sql			

RF-06 - Actualización masiva de conductores inactivos			
Tipo: FUNC	Prioridad: Should	Estado: IMPLEMENTED	
Módulo: Backend Acceso a datos: SP			
Descripción: El sistema deberá actualizar periódicamente el estado de las cuentas de conductor inactivos mediante una operación masiva y dejar traza de auditoría.			
Criterio de aceptación: La operación masiva cambia únicamente usuarios que cumplen el criterio de inactividad (≥6 meses), conserva los demás registros y guarda un registro en la tabla de auditoría.			
Verificación:	Prueba de Integración	(AuthControllerTest.java)	SQL:
db/procs/sp_actualizar_usuarios_inactivos.sql			

RF-07 - Validación cruzada de consistencia de reglas de escenario			
Tipo: FUNC	Prioridad: Must	Estado: IMPLEMENTED	
Módulo: Backend Acceso a datos: SP			
Descripción: El sistema deberá validar que las configuraciones de los escenarios no contengan reglas contradictorias antes de su guardado.			
Criterio de aceptación: La validación de escenario detecta configuraciones inconsistentes y acepta configuraciones válidas.			
Verificación:	Prueba Unitaria	(EscenarioServiceTest.java)	SQL:
db/procs/sp_validar_escenario.sql			

RF-08 - Generación de código secuencial único para certificado			
Tipo: FUNC	Prioridad: Must	Estado: IMPLEMENTED	
Módulo: Backend Acceso a datos: SP			
Descripción: El sistema deberá asignar un código de verificación único y secuencial a cada certificado generado.			
Criterio de aceptación: Cada certificado generado recibe un código secuencial único y reproducible dentro de la transacción usando <code>sp_generar_codigo_certificado</code> .			
Verificación:	Prueba Unitaria	(SimulacionServiceTest.java)	SQL:
db/procs/sp_generar_codigo_certificado.sql			

RF-09 - Evaluación Automática de Comportamiento Vial (IA)
Tipo: FUNC Prioridad: Must Estado: TESTED
Módulo: Motor IA Acceso a datos: CRUD-ORM
Descripción: El sistema deberá evaluar las decisiones del usuario interactuando con un modelo de IA externo (OpenAI) o un motor de reglas local de respaldo, registrando el identificador del modelo utilizado.
Criterio de aceptación: El sistema clasifica el comportamiento vial y guarda la decisión con el identificador del modelo utilizado (IA, IA_LOCAL u OPENAI).
Verificación: Prueba de Integración (FeedbackServiceTest.java) SQL: No aplica

RF-10 - Generación de Retroalimentación Textual (IA)
Tipo: FUNC Prioridad: Must Estado: TESTED
Módulo: Motor IA Acceso a datos: CRUD-ORM
Descripción: El sistema deberá generar retroalimentación textual y recomendaciones basadas en las acciones del conductor durante la simulación.
Criterio de aceptación: Todo comportamiento vial evaluado como incorrecto o subóptimo genera una recomendación de mejora registrada en la base de datos.
Verificación: Prueba de Integración (FeedbackServiceTest.java) SQL: No aplica

RF-11 - Reglas de Negocio: Certificación y Puntaje
Tipo: FUNC Prioridad: Must Estado: IMPLEMENTED
Módulo: Backend Acceso a datos: SP
Descripción: El sistema deberá emitir un certificado si y solo si el puntaje calculado en la simulación es mayor o igual a 80 puntos.
Criterio de aceptación: Las simulaciones con puntaje ≥ 80 disparan la generación de certificado, de lo contrario se marcan como fallidas.
Verificación: Análisis de Inspección (sp_calcular_puntaje.sql) SQL: db/procs/sp_calcular_puntaje.sql

RF-12 - Persistencia de Entidades Secundarias
Tipo: FUNC Prioridad: Should Estado: IMPLEMENTED
Módulo: Database Acceso a datos: CRUD-ORM
Descripción: El sistema deberá registrar y administrar de manera consistente las Reglas de Tránsito, Infracciones, Eventos Viales, Decisiones y Recompensas generadas durante las prácticas.
Criterio de aceptación: Las 5 entidades de dominio (ReglaTransito, Infraccion, EventoVial, Decision, Recompensa) persisten su estado relacional sin errores de integridad.
Verificación: Prueba de Integración (SimulacionIntegracionTest.java) SQL: No aplica

4. Requisitos no funcionales

RNF-01 - Latencia de lectura en caché caliente p95 $\leq$ 200ms
Tipo: PERF Prioridad: Must Estado: VERIFIED
Módulo: API Acceso a datos: No aplica
Descripción: El sistema deberá responder a consultas cacheadas con un percentil 95 de latencia no mayor a 200 milisegundos bajo una carga de 50 usuarios virtuales concurrentes durante 30 segundos.
Criterio de aceptación: La corrida k6 reporta p95 de caché caliente $\leq$ 200 ms y tasa de error inferior a 1 %. Se conserva la salida cruda NDJSON como evidencia en el repositorio.
Verificación: Prueba de Rendimiento (scripts/k6/load-test.js) SQL: No aplica

RNF-02 - Latencia de lectura en frío p95 $\leq$ 500ms
Tipo: PERF Prioridad: Must Estado: VERIFIED
Módulo: API Acceso a datos: No aplica
Descripción: El sistema deberá responder a consultas sin caché previo con un percentil 95 de latencia no mayor a 500 milisegundos bajo carga de 50 VUs por 30s.
Criterio de aceptación: La corrida k6 reporta p95 de lectura fría $\leq$ 500 ms. Se conserva la salida cruda NDJSON.
Verificación: Prueba de Rendimiento (scripts/k6/load-test.js) SQL: No aplica

RNF-03 - Contraseñas hashheadas con BCrypt y factor de costo $\geq$ 10
Tipo: SEC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: No aplica
Descripción: El sistema deberá hashear todas las contraseñas utilizando el algoritmo BCrypt con un factor de costo mínimo de 10.
Criterio de aceptación: Las contraseñas se almacenan con BCrypt y el sistema genera hashes con el prefijo correcto correspondiente al factor de costo configurado.
Verificación: Prueba de Integración (AuthServiceTest.java) SQL: No aplica

RNF-04 - Cookie de token con HttpOnly y SameSite=Strict
Tipo: SEC Prioridad: Must Estado: VERIFIED
Módulo: Backend Acceso a datos: No aplica
Descripción: El sistema deberá enviar el token de sesión a través de una cookie HTTP configurada con los flags HttpOnly y SameSite=Strict.
Criterio de aceptación: La cookie de respuesta incluye HttpOnly y SameSite=Strict en cualquier entorno.
Verificación: Prueba de Integración (AuthControllerTest.java) SQL: No aplica

RNF-05 - Ausencia total de SQL dinámico en procedimientos
Tipo: SEC Prioridad: Must Estado: VERIFIED
Módulo: Database Acceso a datos: No aplica
Descripción: El sistema deberá prevenir la inyección SQL en la base de datos evitando el uso de SQL dinámico inseguro.
Criterio de aceptación: El análisis automático no detecta concatenación de cadenas dentro de bloques de ejecución dinámica (EXECUTE o format).
Verificación: Análisis Estático (scripts/audit-sql-dynamic.sh) SQL: No aplica

RNF-06 - Escala de Usabilidad del Sistema SUS $\geq$ 68 puntos
Tipo: USAB Prioridad: Must Estado: VERIFIED
Módulo: Frontend Acceso a datos: No aplica
Descripción: El sistema deberá ofrecer una interfaz altamente usable, alcanzando un puntaje SUS sobre la media estándar de la industria.
Criterio de aceptación: La evaluación empírica SUS con al menos 15 participantes alcanza una puntuación media calculada igual o superior a 68.
Verificación: Prueba de Usabilidad (docs/mediciones/sus/sus-analysis.md) SQL: No aplica

RNF-07 - Degradación controlada (Fallback) del proveedor de IA
Tipo: SEC Prioridad: Should Estado: VERIFIED
Módulo: Backend Acceso a datos: No aplica
Descripción: El sistema deberá continuar operando mediante un motor de reglas local si el proveedor externo de IA (OpenAI) supera los 2 segundos de latencia o no está disponible.
Criterio de aceptación: Ante indisponibilidad del servicio OpenAI, el sistema hace fallback al modelo local sin fallar la simulación.
Verificación: Prueba de Integración (FeedbackServiceTest.java) SQL: No aplica

RNF-08 - Privacidad de Datos y Minimización
Tipo: SEC Prioridad: Must Estado: TESTED
Módulo: Backend Acceso a datos: No aplica
Descripción: El sistema deberá evitar enviar Información de Identificación Personal (PII) al proveedor externo de Inteligencia Artificial.
Criterio de aceptación: Solo se envían las decisiones y el contexto del escenario a la API de OpenAI, excluyendo nombres, correos o IDs directos.
Verificación: Prueba Unitaria (FeedbackIaExternaServiceTest.java) SQL: No aplica

RNF-09 - Accesibilidad y Compatibilidad (WCAG 2.2)
Tipo: USAB Prioridad: Should Estado: TESTED Módulo: Frontend Acceso a datos: No aplica Descripción: El sistema deberá cumplir con las directrices de accesibilidad web WCAG 2.2 nivel AA. Criterio de aceptación: La aplicación puede ser navegada completamente con el teclado y cuenta con un ratio de contraste adecuado validado por Lighthouse. Verificación: Análisis Estático Automático (Lighthouse) SQL: No aplica
RNF-10 - Capacidad y Volumetría
Tipo: PERF Prioridad: Should Estado: TESTED Módulo: Arquitectura Acceso a datos: No aplica Descripción: El sistema deberá soportar hasta 200 usuarios concurrentes y retener el histórico de prácticas de simulaciones por al menos 5 años. Criterio de aceptación: El diseño de la base de datos permite retención a largo plazo y las pruebas de carga demuestran estabilidad con la concurrencia objetivo. Verificación: Prueba de Carga (scripts/k6/load-test.js) SQL: No aplica

5. Historias de usuario (Casos de Uso)

US-01 - Registro autónomo de conductor
Actor principal: Conductor
Flujo Principal: 1. El conductor navega al formulario de registro. 2. Completa los campos obligatorios. 3. El sistema valida los datos y registra al conductor en la base de datos. 4. El sistema notifica el éxito de la operación.
Excepciones: Si el correo ya existe, el sistema muestra un error y detiene el proceso.
Verificación: Prueba de Componente E2E (app.component.spec.ts) SQL: No aplica

US-02 - Inicio de sesión seguro y obtención de cookie
Actor principal: Conductor / Administrador
Flujo Principal: 1. El usuario ingresa credenciales en el login. 2. El sistema verifica el hash BCrypt. 3. El sistema emite un token JWT como cookie. 4. El usuario es redirigido a su panel de control correspondiente.
Excepciones: Credenciales incorrectas emiten un error "No Autorizado"(401).
Verificación: Prueba de Componente E2E (app.component.spec.ts) SQL: No aplica

US-03 - Exploración interactiva de escenarios viales
Actor principal: Conductor
Flujo Principal: 1. El conductor solicita la lista de escenarios. 2. El sistema recupera los escenarios habilitados. 3. El sistema pagina y muestra los escenarios con sus configuraciones de clima y dificultad.
Excepciones: Si no hay escenarios disponibles, se muestra un aviso de "Catálogo vacío".
Verificación: Prueba de Componente E2E (app.component.spec.ts) SQL: No aplica

US-04 - Ejecución de simulación y captura de decisiones
Actor principal: Conductor
Flujo Principal: 1. El conductor inicia un escenario. 2. Toma decisiones ante eventos viales. 3. El motor IA evalúa cada decisión. 4. El sistema calcula y consolida los resultados invocando un procedimiento almacenado.
Excepciones: Si ocurre una desconexión, la simulación se invalida o se reanuda dependiendo del estado.
Verificación: Prueba de Componente E2E (app.component.spec.ts) SQL: db/procs/sp_reporte_simulacion.sql

US-05 - Visualización de métricas y feedback en dashboard
Actor principal: Conductor
Flujo Principal: 1. El conductor accede al Dashboard. 2. El sistema recupera las métricas calculadas. 3. Se presenta el promedio global, simulaciones recientes y sugerencias de mejora.
Excepciones: Datos insuficientes muestran marcadores por defecto (NA).
Verificación: Prueba de Componente E2E (app.component.spec.ts) SQL: db/procs/sp_calcular_promedio_usuario.sql

US-06 - Descarga de certificado con código secuencial**Actor principal:** Conductor

Flujo Principal: 1. El conductor aprueba una simulación (¿= 80 pts). 2. El sistema genera un certificado invocando la base de datos. 3. El conductor visualiza el certificado y lo exporta en formato inmutable.

Excepciones: Si la práctica no fue aprobatoria, el botón de descarga se mantiene oculto.

Verificación: Prueba de Componente E2E (app.component.spec.ts) **SQL:** db/procs/sp_generar_codigo_certificado.sql

6. Criterios de aceptación de la versión

- **AC-01.** El sistema completo se construye y levanta desde una clonación limpia mediante `make all`.
- **AC-02.** Las pruebas automatizadas superan la cobertura global de 70% y el análisis estático no reporta vulnerabilidades graves.
- **AC-03.** Los procedimientos almacenados están versionados, catalogados, parametrizados y trazados.
- **AC-04.** Cinco corridas k6 cumplen los umbrales frío y caliente; Lighthouse y ZAP conservan evidencia reproducible.
- **AC-05.** El despliegue público dispone de healthcheck, runbook, backup y cuenta demo no administrativa.
- **AC-06.** El repositorio contiene documentación, datos, metadatos y etiqueta v1.0.0 coherentes con el commit declarado.

Fin de la especificación